ClearPay - Engineering Report

BuildStart Software Engineering Intern challenge | AI-assisted prototype | 29 September 2026

1. The design: evidence before approval

ClearPay is a single-merchant payment-review prototype. Its central assumption is that a customer-supplied receipt is a claim, not independent proof of receipt of funds. It combines local image extraction, a server-owned order, authenticated bank-notification evidence and prior transaction allocations. Every submission receives APPROVED, REJECTED or NEEDS VERIFICATION, together with an internal reason, a next action and a separate customer-safe reply.

The application uses Python/FastAPI, SQLite, CPU-based RapidOCR and a small HTML/CSS/JavaScript interface. A pure decision function keeps policy independent from storage and transport. This makes the important rules easy to inspect, test and modify during an interview. The operator dashboard separates submitted values from system conclusions and displays the original image, expected amount, extracted reference, bank message, related payments and audit history. Operators can add notes or reject with a reason; they cannot bypass evidence requirements with a manual approval button.

2. Linking a payment to its order

An order receives an unpredictable server-generated PAY- code. The customer is asked to enter that code in the transfer memo. Automatic approval requires the same code in independent bank evidence, alongside a matching bank reference, full receiving account, exact LKR amount and plausible transaction time. The slip memo alone is insufficient. Several customers can pay the same amount at similar times; matching amount and time therefore never establishes ownership.

This attribution mechanism depends on the bank notification carrying a reliable memo. If the notification omits the memo, masks the receiving account, lacks a transaction timestamp or uses an unsupported format, ClearPay keeps the payment in verification. The short masked SMS in the challenge would not meet this policy. A production bank adapter could use another independently verified ownership link; it must not infer one from amount similarity. WhatsApp customer identity and order access are outside the operator-only prototype.

Money is parsed as decimal text and stored as integer minor units, avoiding binary floating-point comparisons. The policy allows five minutes of timestamp skew and a two-day order payment window. Uncertain extraction is checked before field mismatch rejection so weak OCR does not turn a guess into an accusation.

3. Duplicate and reused payments

The raw image SHA-256 identifies byte-identical submissions and caches OCR. It is not expected to survive crops or new screenshots. A second layer identifies the transaction by receiving account and bank reference, preventing an already accepted transfer from funding another order even when the pixels change. A missing or unreadable reference causes review. Repeated references across bank history are conservatively blocked; production requires bank-specific identity semantics.

Approval, transaction allocation and order payment state are committed together under an immediate SQLite write transaction. Unique database constraints protect transaction, order, payment and bank-message allocation. Two concurrent requests therefore cannot both consume the same transfer. Idempotency keys distinguish a harmless transport retry from a new submission: identical retries return the original response; changed content with the same key is rejected.

4. Suspicious slips and bank SMS

ClearPay does not claim to authenticate a screenshot from its visual appearance. It detects inconsistencies, weak extraction, reused transaction references, wrong account or amount, old transfers and missing independent evidence. Conflicting slip and SMS details require review rather than a fabricated certainty score. OCR confidence is used only as a conservative text-reading gate; it is neither a fraud probability nor a calibrated bank-payment accuracy estimate.

The SMS gateway accepts a sender, persistent event ID and raw message. It verifies HMAC-SHA256 over the transport timestamp and exact request body, checks a five-minute transport window, and enforces a configured sender allowlist. Identical event redelivery is harmless; a reused event ID with different content is rejected. Two different events that disagree about the same bank reference create an evidence conflict. Transaction time is read from the message rather than assumed from arrival time.

The optional merchant-phone bridge uses Termux:API inbox output. It filters the bank sender, writes events to a durable SQLite outbox and retries signed HTTPS delivery until acknowledged. The gateway and bridge queue logic were tested; no real Android phone or bank sender was attached. The prototype supports two conservative labelled credit-message variants rather than guessing arbitrary bank formats. A visibly labelled demo simulator exercises the same ingestion logic and is disabled outside demo mode.

A late SMS rechecks pending submissions. A late contradictory record affecting an approved payment creates an urgent incident without silently undoing an allocation or pretending fulfilment is reversible. A signature proves that the configured gateway sent the message; it does not prove bank settlement or prevent a compromised merchant phone from lying. Real deployment therefore needs reconciliation and stronger device controls.

5. Cost decisions and failure handling

There are no paid AI calls. Local CPU OCR runs only for uncached images; one initialized engine is reused. Successful extraction is cached by content hash, while a temporary engine failure is retried on a new submission. A repeated idempotent request bypasses OCR altogether. Low-confidence cases go to human review rather than an expensive model whose confident output still cannot prove that money reached the merchant.

This design trades higher review volume for lower false-approval risk and predictable processing cost. Hosting, CPU, storage, device connectivity and human review are still real costs. A future model fallback would be justified only after measuring a meaningful reduction in review effort without increasing unsafe approvals. Monthly cost should be calculated as fixed infrastructure plus uncached-image processing plus escalations plus review time; no fabricated cloud price or production accuracy estimate is presented.

The application bounds image bytes and decoded dimensions, re-encodes accepted images, escapes displayed text, uses operator sessions and protects the gateway separately. A local database or model error must not become an approval. The current process is synchronous and intentionally small; overload control and robust job retries belong in a production queue. Exact historical decisions and policy versions remain visible through the audit trail.

6. Testing and observed debugging

Testing separates pure decision policy, API persistence, gateway authentication, concurrency, local OCR and browser behavior. Cases include normal payment, underpayment/overpayment by one cent, wrong currency/account, old or future dates, ambiguous or missing SMS, incorrect order ownership, duplicate images, changed-image reuse, retries, invalid uploads and conflicting late evidence. An actual generated PNG is uploaded through OCR, held pending, and approved only after a correctly signed matching SMS. A heavily blurred generated receipt is kept below the reliable-evidence gate.

The first complete run exposed seven failures from an SMS INSERT using seven placeholders for a six-column table. Tracebacks identified the common storage path; explicit column names and six placeholders fixed the affected flows, and all 49 tests then passed. Real OCR next read the label Account as Acc0unt while preserving the number. A narrow label-only parser change recovered the account without rewriting financial values. Finally, a new failure-recovery test proved that temporary OCR outages were cached indefinitely: it expected two engine calls but observed one. Excluding unavailable extraction from reusable cache fixed that regression while preserving successful caching. Detailed before/after evidence is in DEBUGGING.md.

Subsequent runs passed the expanded synthetic test suite, including actual OCR. Browser checks verified sign-in, seeded scenarios, evidence detail, filtering, order creation and delayed notification approval. These results establish the implemented behavior on controlled cases. They do not establish real-bank false-approval, false-rejection or incorrect-match rates. Before deployment, evaluate a consented, labelled multi-bank corpus with adversarial edits and report these outcomes separately from review rate and OCR field accuracy.

7. Limitations and production evolution

The implementation handles one merchant, one receiving account, LKR and selected English labelled formats. It has no live WhatsApp connector, direct bank API, settlement guarantee, visual forgery classifier, refund workflow, instalment reconciliation or two-person manual approval. The simulated cases do not count as real payment verification. A bank that omits the required evidence will generate more review work. High OCR confidence can still be wrong; the conservative field parser and bank corroboration reduce risk but cannot eliminate it.

For 100,000+ monthly submissions, first measure peak arrivals and service times rather than equating monthly volume with throughput. Move to tenant-scoped PostgreSQL constraints and indexed evidence lookup; store images in private object storage with retention rules. Add a durable job queue with bounded OCR workers, retry budgets and a transactional outbox so notification delivery and re-evaluation survive restarts. Preserve allocation invariants even as workers scale.

Replace the shared operator password and in-memory sessions with individual identities, roles and shared session storage. Give each merchant device its own revocable signing key, enforce HTTPS, monitor phone health and verify sender provenance as far as the platform permits. Add statement reconciliation, reversal/chargeback handling, immutable external audit retention, bank-specific versioned parsers and explicit manual-review SLAs. Monitor false approvals, false rejections, incorrect attribution, review rate, backlog age, p95 latency and cost per submission. Production rollout should start with shadow decisions and human confirmation before any automated fulfilment.

References

FastAPI upload documentation: https://fastapi.tiangolo.com/tutorial/request-files/

SQLite transaction semantics: https://www.sqlite.org/lang_transaction.html

RapidOCR: https://github.com/RapidAI/RapidOCR

Termux:API: https://github.com/termux/termux-api